

Reporte de la práctica:

Encendido de LED vía web con Arduino (WiFi)

1. Datos generales

Materia	Desarrollo Sustentable
Práctica	Encendido y apagado del LED 13 vía web (servidor WiFi)
Placa utilizada	Arduino UNO R4 WiFi
Simulador	Tinkercad Circuits (diagrama de conexiones)
Librería	WiFiS3 (incluida en el paquete de placas UNO R4)
	Arduino UNO R4 WiFi, LED rojo, resistencia de 220 Ω , protoboard, cables de conexión, computadora o celular conectado a la misma red WiFi

Diagrama del circuito (conexiones)

	Componente	Conexión
Componentes	Pin digital 13 del Arduino	Terminal de la resistencia de 220 Ω (misma columna del protoboard)
	Otra terminal de la resistencia	Ánodo del LED (+), pata doblada
	Cátodo del LED (-), pata recta	Riel negativo (-) del protoboard
	Riel negativo (-) del protoboard	GND del Arduino

2. Diagrama del circuito

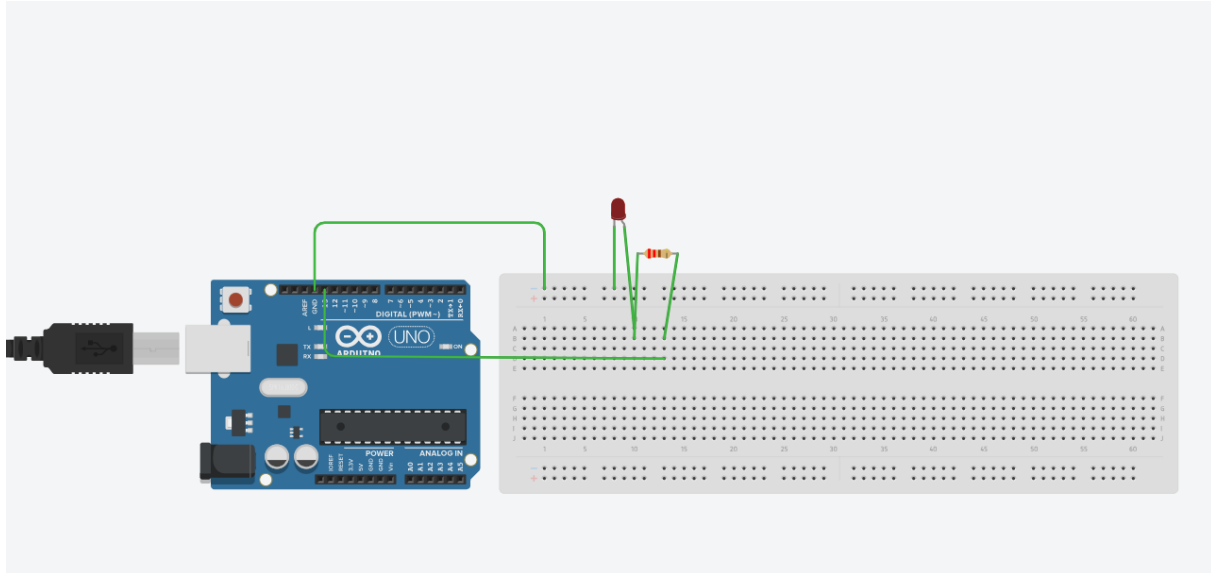


Figura 1. Circuito del LED en el pin 13 armado en Tinkercad Circuits.

3. Objetivo

Controlar el encendido y apagado de un LED conectado al pin digital 13 del Arduino desde un navegador web, utilizando el módulo WiFi integrado de la placa para crear un servidor web dentro de la red local.

4. Código utilizado

```
#include <WiFiS3.h>

const char* ssid = "NOMBRE_DE_TU_RED";
const char* password = "TU_CONTRASEÑA";
const int ledPin = 13;
bool ledState = false;
WiFiServer server(80);

void setup() {
  Serial.begin(9600);
  unsigned long t0 = millis();
  while (!Serial && millis() - t0 < 5000) {
    delay(10);
  }

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW);

  WiFi.begin(ssid, password);
  Serial.print("Conectando a WiFi");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  while (WiFi.localIP() == IPAddress(0, 0, 0, 0)) {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
}
```

```

Serial.print("Servidor listo en: http://");
Serial.println(WiFi.localIP());

server.begin();
}

void loop() {
  WiFiClient client = server.available();
  if (!client) return;

  String request = ""; // primera línea (GET /algo HTTP/1.1)
  String currentLine = "";
  bool firstLineDone = false;

  while (client.connected()) {
    if (client.available()) {
      char c = client.read();

      if (!firstLineDone) {
        request += c;
        if (c == '\n') firstLineDone = true;
      }

      if (c == '\n') {
        if (currentLine.length() == 0) break; // línea en blanco = fin de los headers
        currentLine = "";
      } else if (c != '\r') {
        currentLine += c;
      }
    }
  }

  if (request.indexOf("GET /led/on") >= 0) {
    ledState = true;
    digitalWrite(ledPin, HIGH);
  } else if (request.indexOf("GET /led/off") >= 0) {
    ledState = false;
    digitalWrite(ledPin, LOW);
  }

  client.println("HTTP/1.1 200 OK");
  client.println("Content-Type: text/html");
  client.println("Connection: close");
  client.println();
  client.println("<!DOCTYPE html><html><head>");
  client.println("<meta name='viewport' content='width=device-width, initial-scale=1'>");
  client.println("<style>body{font-family:sans-serif;text-align:center;margin-top:50px;}");
  client.println("<button{font-size:20px;padding:15px 30px;margin:10px;}</style></head><body>");
  client.println("<h1>Control de LED</h1>");
  client.print("<p>Estado actual: <b>");
  client.print(ledState ? "ENCENDIDO" : "APAGADO");
  client.println("</b></p>");
  client.println("<a href='/led/on'><button>Encender</button></a>");
  client.println("<a href='/led/off'><button>Apagar</button></a>");
  client.println("</body></html>");

  delay(1);
  client.stop();
}

```

Nota: el nombre y la contraseña de la red se omiten por seguridad.

5. Funcionamiento

Al iniciar, el programa configura el pin 13 como salida con el LED apagado y conecta la placa a la red WiFi. Una vez que obtiene una dirección IP, la muestra en el Monitor Serial (por ejemplo, `http://192.168.x.x`) e inicia un servidor web en el puerto 80.

Cuando un usuario abre esa dirección en el navegador, el Arduino responde con una página HTML titulada "Control de LED", que muestra el estado actual del LED y dos botones. El botón Encender envía la petición `GET /led/on`, que pone el pin 13 en HIGH; el botón Apagar envía `GET /led/off`, que lo pone en LOW. Después de cada petición, la página se actualiza con el nuevo estado (ENCENDIDO o APAGADO) y se cierra la conexión con el cliente.

La resistencia de 220 Ω limita la corriente que circula por el LED para evitar que se dañe.

6. Relación con el desarrollo sustentable

El control remoto de dispositivos es la base de los sistemas domóticos e IoT. Poder encender y apagar una carga a distancia permite apagar luces o equipos olvidados sin desplazarse, lo que reduce el consumo innecesario de energía. Además, el uso de un LED como indicador ya representa una opción de bajo consumo frente a la iluminación tradicional.

7. Conclusión

Se logró controlar el LED del pin 13 desde el navegador de una computadora o celular conectado a la misma red. La práctica permitió comprender cómo el Arduino puede funcionar como servidor web, recibir peticiones HTTP y responder con una página HTML, así como su potencial para aplicaciones de ahorro energético mediante el control remoto de dispositivos.